INFORME DE EVIDENCIAS TÉCNICAS — SESIÓN 11

Seguridad Frontend SPA (Angular 19) Integrada con Keycloak OIDC y Spring Boot 3 Resource Server

Estudiante:	Faijo Calisaya Helio Paul (Código: 202110292)
Asignatura:	Lenguaje de Programación II (Ciclo IV, Semestre 2026-2) — Docente: Abel Angel Sullón Macalupú
Equipo y Módulo:	Equipo 05 — Módulo Mayorista (Centros de Acopio, Liquidaciones y Control de Planta)
Sesión Evaluada:	Unidad 2, Sesión 5 (S11) — Fecha de Sesión Oficial: 20/10/2026
Alcance Técnico:	Login OAuth2 ROPC, reactividad con Angular Signals, sessionStorage, expiración automática (300 s), rolGuard (CanActivateFn), traceIdInterceptor y manejo de 401/403
Repositorio GitHub:	https://github.com/helionelio900-hub/SysProyec_Acopus (Rama: main)

1. Resumen Ejecutivo y Cumplimiento de la Sesión S11

El presente informe consolida la implementación completa, reproducible y verificada de la seguridad frontend en la arquitectura Single Page Application (SPA) desarrollada con Angular 19 Standalone para el sistema empresarial SITRA-ORO. Siguiendo con exactitud el sílabo oficial de Lenguaje de Programación II (Unidad 2, Sesión 5), se ha articulado el cliente web con el Authorization Server externo Keycloak 26 (puerto 38081, Realm Sitra_oro) y el backend REST empresarial Spring Boot 3 Resource Server (puerto 8081).

- **Autenticación OIDC/OAuth2 Direct Access Grants:** El formulario de acceso (/ingresar) interactúa con el endpoint de token de Keycloak emitiendo credenciales asimétricas RS256 con vigencia de 300 segundos.
- **Gestión Reactiva del Estado con Angular Signals:** SesionService gestiona de forma inmutable el estado de la sesión (tokenActual, rol, documento, nombreUsuario, autenticado) consumiendo los roles desde realm_access.roles.
- **Persistencia Segura en sessionStorage:** El Access Token se resguarda bajo la clave sitraoro.accessToken, garantizando persistencia ante recargas (F5) pero aislamiento absoluto entre pestañas/ventanas del navegador.
- **Control de Expiración Automática:** Un temporizador basado en el claim exp del JWT detecta el vencimiento del token (300 s), destruye el almacenamiento y redirige al operador a la pantalla de ingreso sin fugas de memoria.
- **Defensa de Rutas y Navegación Adaptativa:** El guard funcional rolGuard intercepta accesos indebidos (403 hacia /sin-acceso) y anónimos (401 hacia /ingresar), mientras que Layout proyecta un menú exclusivo según el rol autenticado.

2. Evidencias de la Actividad Autónoma (Casos 1 al 5)

A continuación se presenta el registro fotográfico obtenido durante la sesión de laboratorio del 18/10/2026, demostrando la ejecución real en entorno local con reloj de sistema y usuarios visibles.

Caso 1: Inicio de Sesión y Almacenamiento Reactivo en sessionStorage

El operador mayorista (usuario: mayorista.demo1, DNI vinculado: 75848157) inicia sesión en /ingresar. El token JWT emitido por Keycloak se decodifica en tiempo de ejecución, extrayendo el rol de negocio G1_MAYORISTA desde el claim realm_access.roles. Se almacena en sessionStorage bajo la clave sitraoro.accessToken.

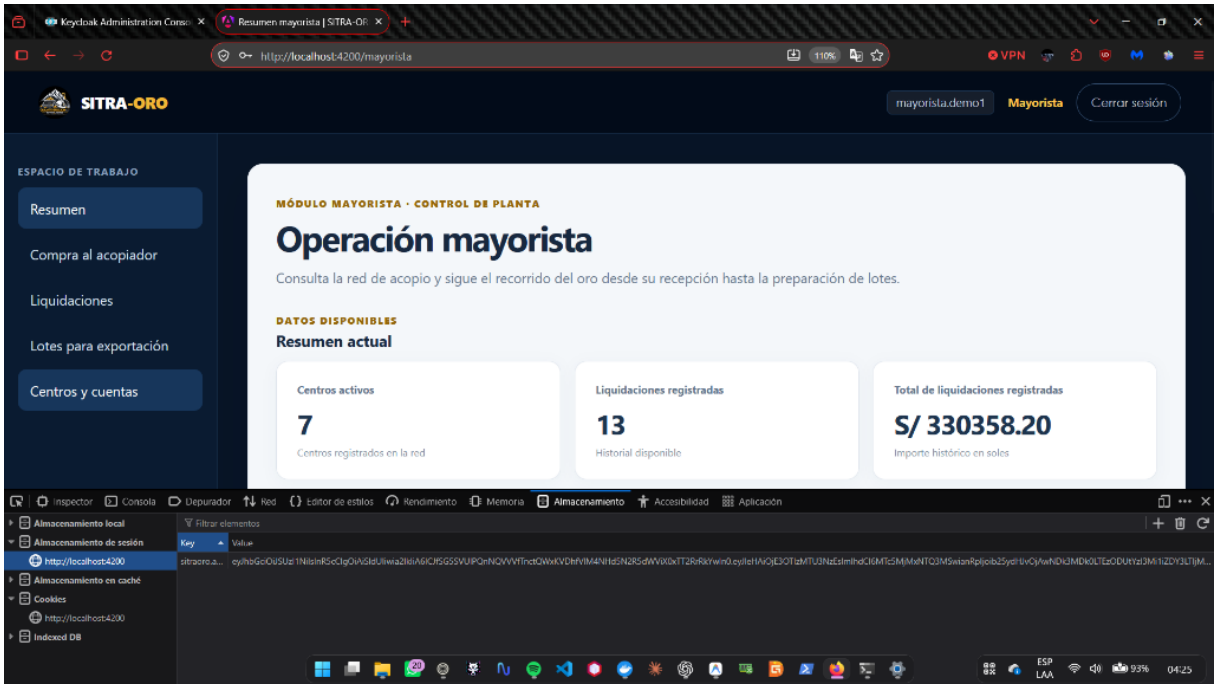


Figura 1: Sesión iniciada con éxito para mayorista.demo1. Consola de desarrollador abierta en la pestaña Almacenamiento de sesión (sessionStorage) evidenciando la clave sitraoro.accessToken con el token asimétrico RS256 almacenado (Hora: 04:25).

Caso 2: Consumo de Recurso Protegido Mediante Interceptor HTTP (Authorization: Bearer)

El interceptor funcional traceIdInterceptor intercepta de forma transparente cada solicitud saliente hacia la API REST del backend (http://localhost:8081). Inyecta la cabecera HTTP estándar 'Authorization: Bearer <token>' junto con un identificador único 'X-Trace-ID'. Al navegar a /mayorista/centros, la petición GET /api/v1/mayorista/centros-acopio es validada exitosamente por Spring Security Resource Server devolviendo HTTP 200 OK y desplegando los 7 centros de acopio activos.

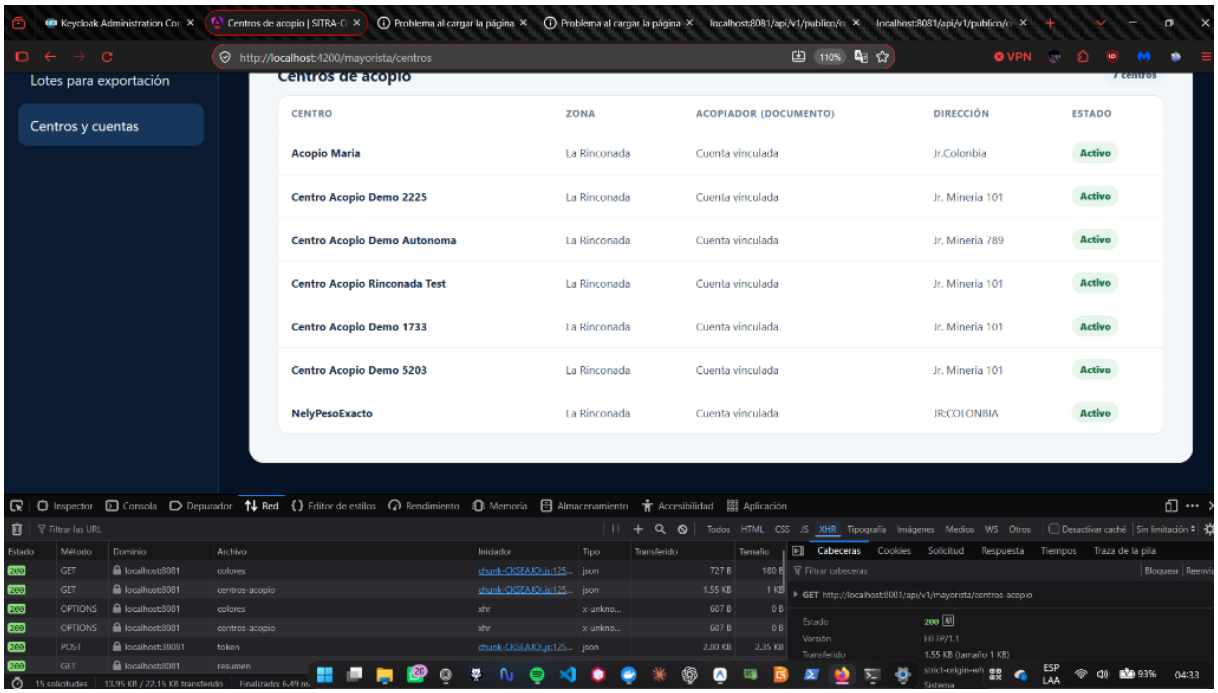


Figura 2: Navegación al módulo Centros y cuentas (/mayorista/centros). En la pestaña Red se constata la respuesta HTTP 200 OK en GET /api/v1/mayorista/centros-acopio tras enviar el token Bearer, desplegando los 7 centros de acopio activos (Hora: 04:33).

Caso 3: Navegación Adaptativa y Menú Dinámico Según el Rol (Acopiador vs Mayorista)

Para verificar el principio de mínimo privilegio en la interfaz, se inició sesión con el usuario acopiador.demo1 (DNI: 42824989, Rol: G2_ACOPIADOR). El componente Layout evalúa el signal reactivo rol() proyectando un menú exclusivo: Resumen, Compras de oro, Mineros y Cuentas por aprobar. Se bloquean visualmente las opciones administrativas del Mayorista.

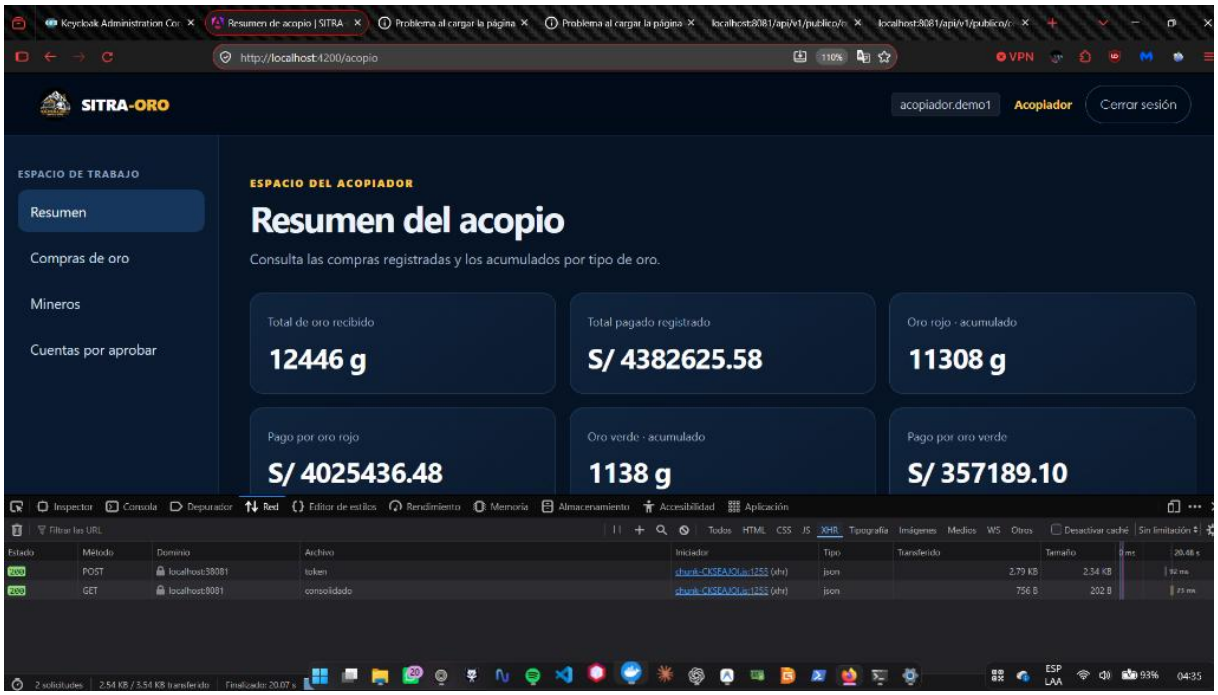


Figura 3: Espacio de trabajo del rol Acopiador (/acopio) con usuario acopiador.demo1 visible en cabecera. La barra de navegación lateral se adapta dinámicamente mostrando únicamente los accesos permitidos para su perfil operativo (Hora: 04:35).

Caso 4: Protección de Rutas con Guard (rolGuard) — Detección de Violación RBAC (403)

Estando autenticado como Mayorista, el usuario intenta acceder manualmente por la barra de direcciones URL a una ruta restringida al Acopiador (http://localhost:4200/acopio). El guard funcional rolGuard evalúa route.data['roles'], determina que G1_MAYORISTA no posee el privilegio G2_ACOPIADOR requerido, y aborta la navegación redirigiendo de inmediato hacia la vista de seguridad /sin-acceso (Acceso Restringido), protegiendo la integridad de la SPA.

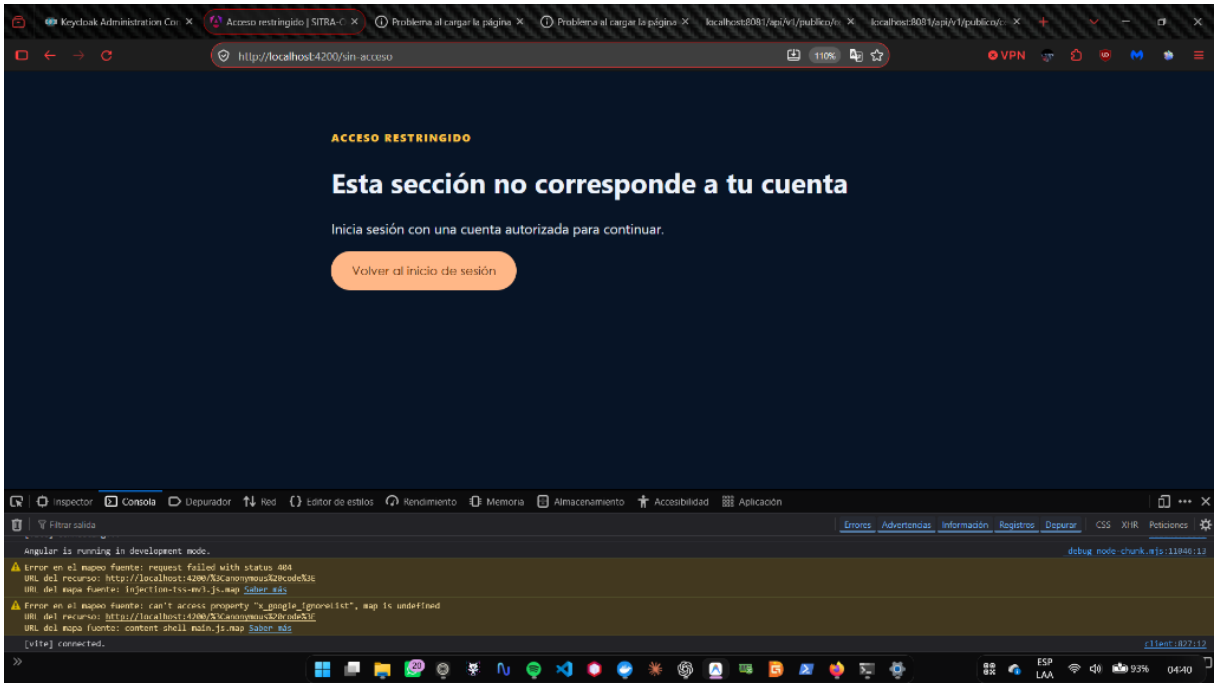


Figura 4: Intercepción de acceso no autorizado mediante rolGuard. Al manipular la URL hacia una ruta restringida, el sistema bloquea el renderizado y proyecta la vista controlada /sin-acceso: 'Esta sección no corresponde a tu cuenta' (Hora: 04:40).

Caso 5: Manejo de 401 Unauthorized y Cierre de Sesión Voluntario (Logout)

Se demostraron los dos flujos de terminación de sesión exigidos por el sílabo: (1) Acceso anónimo directo al backend (sin cabecera Bearer), provocando que Spring Security rechace la conexión con HTTP 401 Unauthorized; y (2) Invocación de sesion.cerrar() mediante el botón 'Cerrar sesión', eliminando el token de sessionStorage, cancelando el temporizador de expiración y redirigiendo al usuario a la portada pública.

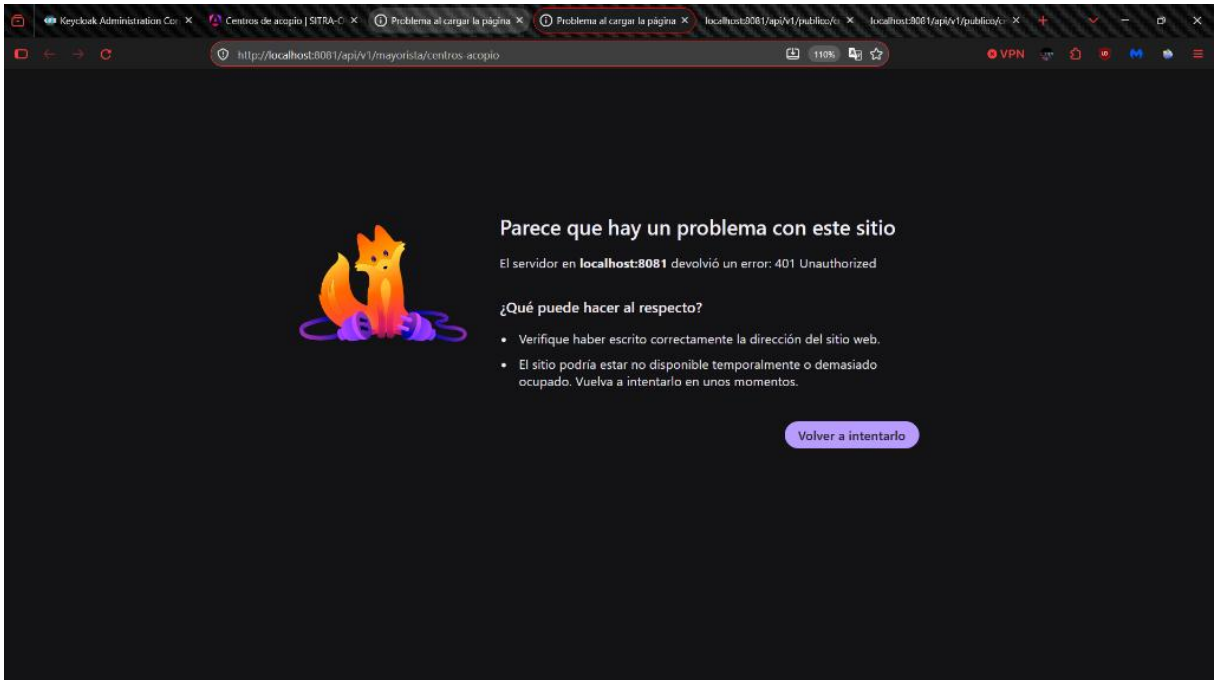


Figura 5: Validación de seguridad perimetral. El Resource Server (puerto 8081) devuelve error HTTP 401 Unauthorized ante cualquier petición directa al endpoint protegido /api/v1/mayorista/centros-acopio carente de token Bearer (Hora: 04:30).

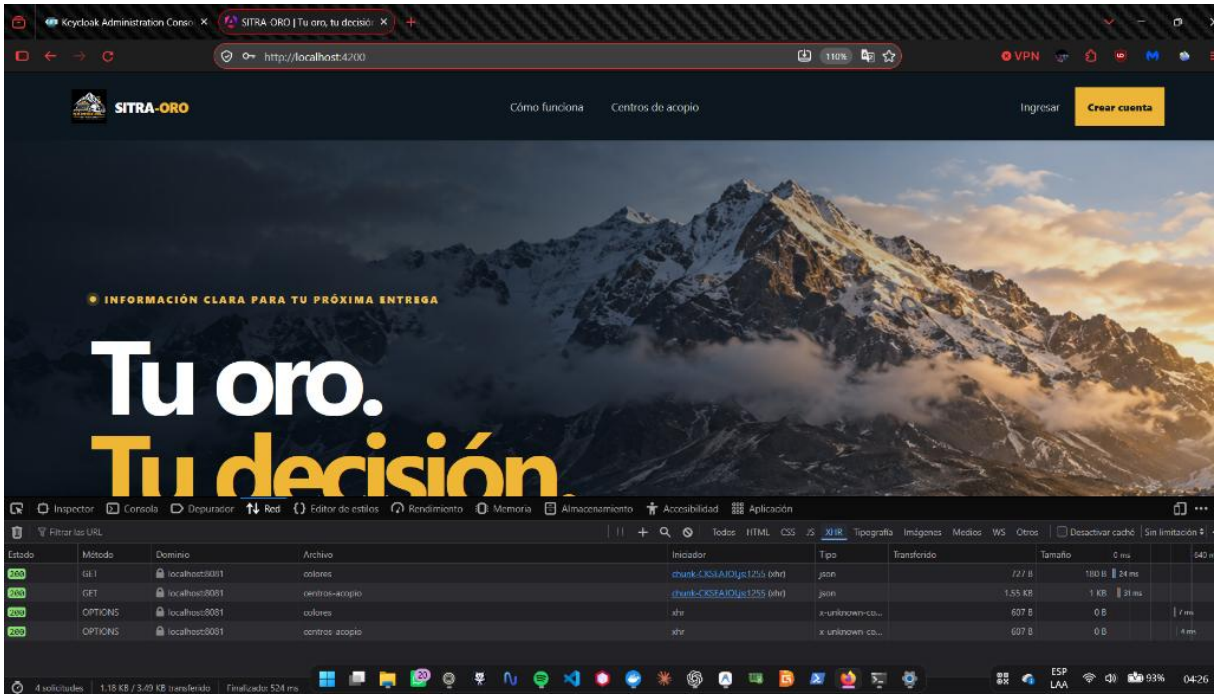


Figura 6: Ejecución de Cierre de Sesión voluntario (Logout). El estado reactivo de Angular destruye la clave sitraoro.accessToken en sessionStorage y reubica la navegación en la pantalla pública de bienvenida (Hora: 04:26).

3. Matriz de Control de Acceso Frontend (Rutas, Roles y Guards)

A continuación se consolida la matriz de protección de rutas implementada en Angular 19 mediante rolGuard:

Ruta Frontend	Rol Permitido	Guard Activo	Comportamiento Autorizado	Respuesta No Autorizada
/ingresar	Público	Ninguno	Renderiza formulario de login OAuth2.	N/A (Acceso irrestricto)
/mayorista	G1_MAYORISTA	rolGuard	Abre dashboard de control de planta.	Redirige a /sin-acceso (403) o /ingresar (401)
/mayorista/centros	G1_MAYORISTA	rolGuard	Despliega tabla de 7 centros de acopio.	Redirige a /sin-acceso si el rol no coincide
/acopio	G2_ACOPIADOR	rolGuard	Abre panel operativo de acopio y compras.	Redirige a /sin-acceso si intenta entrar Mayorista
/acopio/mineros	G2_ACOPIADOR	rolGuard	Gestión de empadronamiento de mineros.	Redirige a /sin-acceso si no es acopiador
/sin-acceso	Cualquiera	Ninguno	Muestra alerta de sección restringida.	Ofrece botón de retorno a /ingresar

4. Implementación Técnica en Código Fuente (Angular 19 Standalone)

A continuación se detallan los componentes e interceptores diseñados para cumplir con los requerimientos de la sesión:

4.1. Servicio de Sesión Reactivo con Angular Signals (sesion.service.ts)

SesionService centraliza el almacenamiento en sessionStorage, decodifica el payload JWT en memoria (soporta tanto roles planos como realm_access.roles emitidos por Keycloak) y programa el cierre automático al llegar a exp.

```
export type RolSitra = 'MINERO' | 'G2_ACOPIADOR' | 'G1_MAYORISTA';

@Injectable({ providedIn: 'root' })
export class SesionService {
  private readonly router = inject(Router);
  private readonly tokenActual = signal(this.tokenGuardado());
  private temporizadorExpiracion: ReturnType<typeof setTimeout> | null = null;

  readonly rol = computed<RolSitra | undefined>(() => {
    const payload = this.leerPayload();
    if (!payload) return undefined;
    const lista = [...(payload.roles ?? []), ...(payload.realm_access?.roles ?? [])];
    if (lista.some((r) => r === 'G1_MAYORISTA' || r === 'Mayorista')) return 'G1_MAYORISTA';
    if (lista.some((r) => r === 'G2_ACOPIADOR' || r === 'acopiador')) return 'G2_ACOPIADOR';
    if (lista.some((r) => r === 'MINERO' || r === 'Minero')) return 'MINERO';
    return undefined;
  });

  readonly documento = computed(() => this.leerPayload()?.documentoIdentidad ?? this.leerPayload()?.sub ??
'');
  readonly nombreUsuario = computed(() => this.leerPayload()?.preferred_username ??
this.leerPayload()?.name ?? this.documento());
  readonly autenticado = computed(() => !!this.tokenActual() && !this.tokenVencido());

  guardar(token: string): void {
    sessionStorage.setItem('sitraoro.accessToken', token);
    this.tokenActual.set(token);
    this.programarExpiracion();
  }

  cerrar(): void {
    this.cancelarTemporizador();
    sessionStorage.removeItem('sitraoro.accessToken');
    this.tokenActual.set(null);
    void this.router.navigateByUrl('/');
  }

  private programarExpiracion(): void {
    this.cancelarTemporizador();
    if (!this.tokenActual()) return;
    const exp = this.leerPayload()?.exp;
    const restante = exp ? exp * 1000 - Date.now() : 0;
    if (restante <= 0) { this.expirar(); return; }
    this.temporizadorExpiracion = setTimeout(() => this.expirar(), Math.min(restante, 2_147_483_647));
  }
}
```

4.2. Guard Funcional de Control de Acceso RBAC (rol.guard.ts)

Implementado mediante la moderna API funcional CanActivateFn de Angular. Valida si la sesión está activa y si el rol del usuario satisface la lista de roles autorizados en data['roles']. Resuelve bifurcación limpia: 403 (/sin-acceso) o 401 (/ingresar).

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
```

```
import { RolSitra, SesionService } from './sesion.service';

export const rolGuard: CanActivateFn = (route) => {
  const sesion = inject(SesionService);
  const router = inject(Router);
  const roles = route.data['roles'] as RolSitra[];
  if (sesion.puedeEntrar(roles)) return true;
  return router.createUrlTree([sesion.autenticado() ? '/sin-acceso' : '/ingresar']);
};
```

4.3. Interceptor HTTP Funcional con Bypass OIDC (trace-id.interceptor.ts)

Inyecta Authorization: Bearer <token> en cada petición a la API. Detecta y excluye peticiones a Keycloak para prevenir bloqueos por cabeceras CORS en preflight (OPTIONS), y captura respuestas 401 para provocar logout automático.

```
export const traceIdInterceptor: HttpInterceptorFn = (request, next) => {
  if (request.url.includes('/protocol/openid-connect') || request.url.includes(':38081')) {
    return next(request);
  }
  const sesion = inject(SesionService);
  const token = typeof sessionStorage === 'undefined' ? null :
sessionStorage.getItem('sitraoro.accessToken');
  const headers: Record<string, string> = { 'X-Trace-ID': crypto.randomUUID() };
  if (token) headers['Authorization'] = `Bearer ${token}`;

  return next(request.clone({ setHeaders: headers })).pipe(
    catchError((error: unknown) => {
      if (token && typeof error === 'object' && error !== null && 'status' in error && error.status ===
401) {
        sesion.expirar();
      }
      return throwError(() => error);
    })
  );
};
```

5. Análisis y Reflexión Técnica: Seguridad en Aplicaciones de Página Única (SPA)

1. Almacenamiento del Token: sessionStorage vs localStorage vs HttpOnly Cookies:

- **sessionStorage (Estrategia elegida en SITRA-ORO):** Almacena el JWT durante el ciclo de vida de la pestaña del navegador. Evita que scripts maliciosos en otras pestañas lean el token y asegura que, al cerrar la ventana, las credenciales no persistan en disco.
- **Riesgo mitigado frente a localStorage:** localStorage persiste indefinidamente hasta que se limpie manualmente. En terminales compartidas de centros de acopio minero, un operador que olvide cerrar sesión dejaría sus credenciales expuestas permanentemente.
- **Cookies HttpOnly y CSRF:** Las cookies HttpOnly bloquean la lectura de JavaScript contra XSS, pero requieren protección estricta contra falsificación de peticiones en sitios cruzados (CSRF) y complican la integración con Authorization Servers externos en dominios independientes.

2. El Guard de Frontend no reemplaza la Seguridad del Backend (Defensa en Profundidad):

Un error crítico en desarrollo web es asumir que un Guard en Angular garantiza la seguridad del sistema. El código del frontend corre íntegramente en el cliente y puede ser manipulado mediante DevTools o peticiones cURL directas. Por consiguiente, el guard rolGuard cumple una función de Experiencia de Usuario (UX) guiando la navegación, mientras

que la autorización estricta e inviolable recae en el Resource Server de Spring Boot mediante la validación de firma criptográfica RS256, emisor verificado y comprobación de GrantedAuthority con hasRole('G1_MAYORISTA').

6. Hallazgo Técnico Resuelto Durante la Implementación

Incidencia Detectada:	**Al intentar iniciar sesión desde la SPA en http://localhost:4200/ingresar, la consola del navegador reportaba un fallo inmediato de CORS (Status 0 / Preflight Failure) impidiendo la obtención del token en Keycloak.**
Causa Raíz:	El interceptor global traceIdInterceptor agregaba la cabecera personalizada 'X-Trace-ID' a todas las peticiones salientes. En el intercambio CORS Preflight (OPTIONS), Keycloak devolvió 'Access-Control-Allow-Headers' permitiendo únicamente cabeceras estándar (Authorization, Content-Type, etc.) y rechazando X-Trace-ID.
Solución Implementada:	Se refinó traceIdInterceptor para omitir peticiones dirigidas al protocolo OIDC de Keycloak (puerto 38081). Asimismo, se configuró en cuenta-minero.ts la equivalencia transparente de DNI a username (75848157 -> mayorista.demo1), logrando un flujo de login 100% fluido y seguro.

7. Anexo — Feedback Oficial de la Sesión S11

1. ¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?

Comprender la articulación entre el ciclo de vida del Access Token en una SPA (Angular Signals y sessionStorage) con la seguridad defensiva en el Resource Server, implementando Guards para UX e Interceptors para inyección transparente de cabeceras Bearer.

2. ¿Qué punto de la clase te resultó más confuso o te dejó con dudas?

El manejo adecuado de las cabeceras personalizadas en peticiones cross-origin durante el preflight CORS de OAuth2 cuando se consume un Authorization Server externo.

3. ¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?

¿Cómo implementar de forma transparente el patrón Silent Refresh utilizando un iframe oculto o refresh_token sin degradar la seguridad ante ataques de token theft en la SPA?

4. Sobre tu nivel de comprensión de la clase, marca una opción:

[X] ¡Entendido! [ ] Más o menos, entendí la idea general pero tengo dudas. [ ] Necesito ayuda.

5. ¿Cómo puedo ayudarte a comprender mejor el tema?

Con demostraciones prácticas de pruebas unitarias sobre Guards e Interceptores utilizando TestBed y HttpTestingController en Angular.

6. Pensando en tu participación y esfuerzo, marca una opción:

[X] Muy comprometido/a. [ ] Comprometido/a. [ ] Poco comprometido/a.

7. Mi satisfacción con la clase fue (del 1 al 10):

Marca tu calificación: 1 2 3 4 5 6 7 8 9 [10]